LiDAR Patrol

LiDAR Patrol

1. Course Content
2. Case Demonstration
3. Source Code Analysis

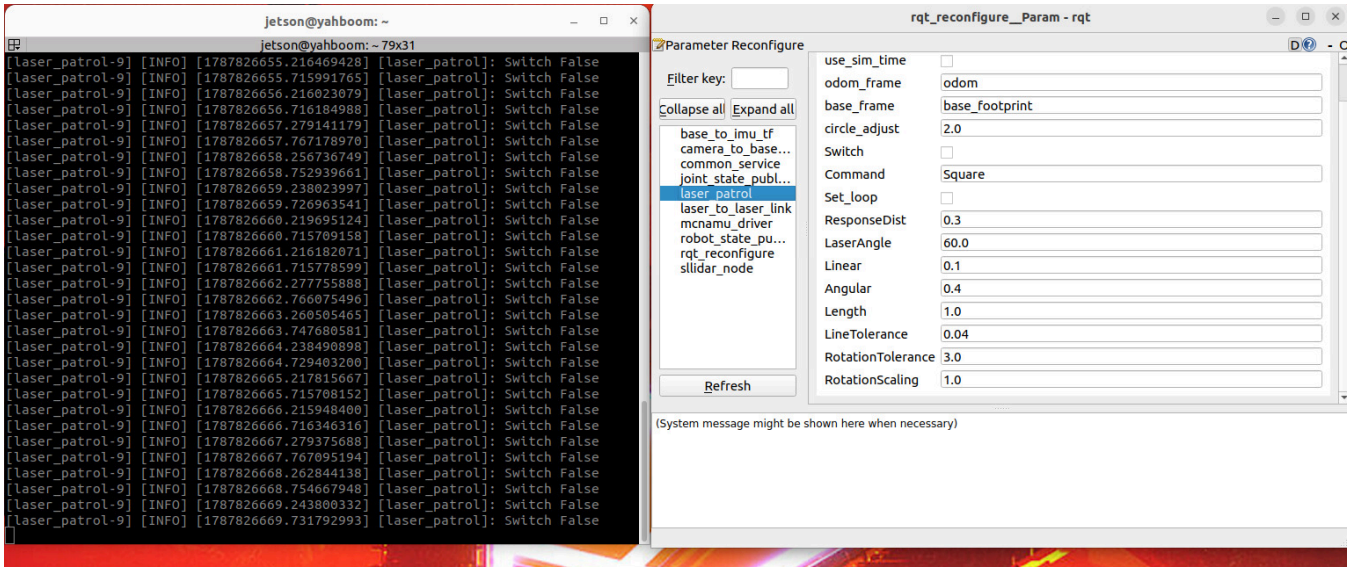
1. Course Content

- Run the example program, set the route to be patrolled in the dynamic parameter tuner, then click start. The car will move according to the set patrol route, and at the same time the LiDAR on the car will scan within the set LiDAR angle and the set obstacle detection distance range to check whether there is an obstacle. If there is an obstacle, it will stop and the buzzer will sound; if there is no obstacle, it will continue patrolling.

2. Case Demonstration

```
ros2 launch yahboomcar_bringup lidar_demo.launch.py demo:=laser_patrol
```

- Click the **laser_patrol** node in the left option bar of the `rqt_reconfigure` panel. If there is no node in the left options right after startup, click **Refresh** below to refresh



- Enter the preset patrol route in 【**Command**】, then check Switch to make the car start patrolling

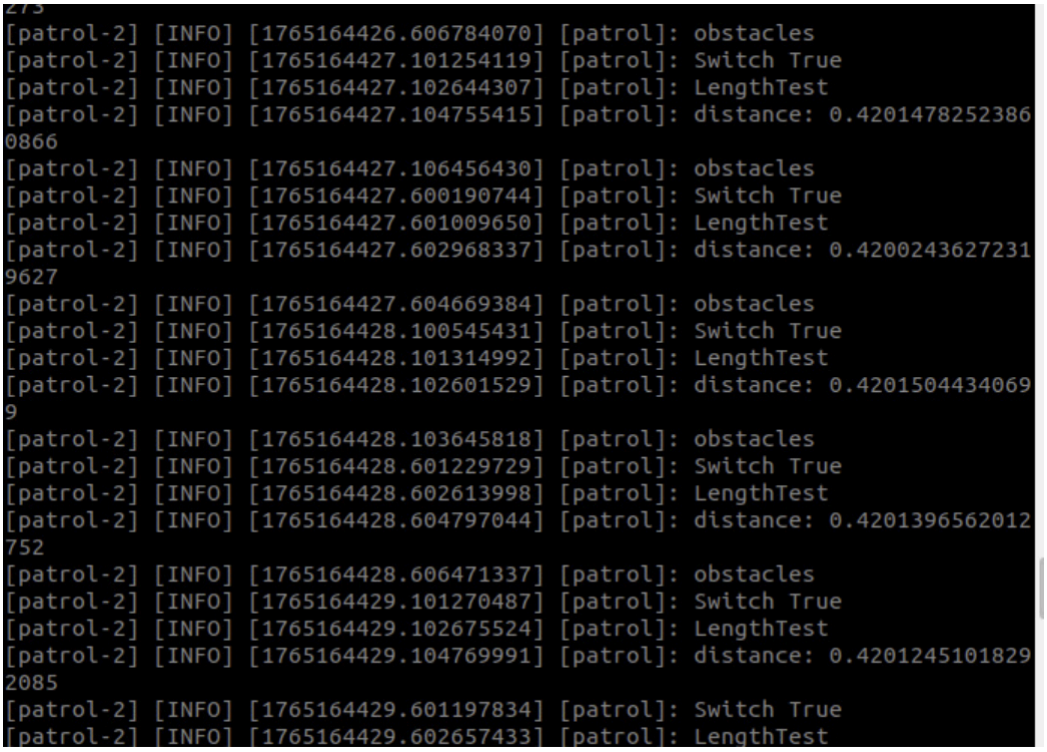


> **Tip**
>
> `rqt_reconfigure` panel parameter description:
>
> - `odom_frame`: the name of the odometry coordinate frame
> - `base_frame`: the name of the base coordinate frame
> - `circle_adjust`: when the patrol route is a circle, this value can be used as a coefficient to adjust the circle's size; see the code for details
> - `Switch`: mode switch
> - `Command`: the patrol route, with the following options: 【LengthTest】- straight-line patrol, 【Circle】- circle patrol, 【Square】- square patrol, 【Triangle】- triangle patrol.
> - `Set_loop`: the switch for re-patrolling; once set, it will continue cycling patrols according to the set route
> - `ResponseDist`: the obstacle detection distance
> - `LaserAngle`: the LiDAR detection angle
> - `Linear`: the linear velocity magnitude
> - `Angular`: the angular velocity magnitude
> - `Length`: the distance of straight-line motion
> - `RotationTolerance`: the allowed tolerance value for rotation error
> - `LineTolerance`: the allowed tolerance value for straight-line motion error

- After setting the route in 【Command】, click Switch to start patrolling. The terminal printout is as follows:

```
[patrol-2] [INFO] [1765164370.602805186] [patrol]: LengthTest
[patrol-2] [INFO] [1765164370.604678533] [patrol]: distance: 0.0270851628480
32567
[patrol-2] [INFO] [1765164371.101304815] [patrol]: Switch True
[patrol-2] [INFO] [1765164371.102707260] [patrol]: LengthTest
[patrol-2] [INFO] [1765164371.104783647] [patrol]: distance: 0.0751071287024
9182
[patrol-2] [INFO] [1765164371.601953276] [patrol]: Switch True
[patrol-2] [INFO] [1765164371.603405194] [patrol]: LengthTest
[patrol-2] [INFO] [1765164371.605899706] [patrol]: distance: 0.1864800882978
9733
[patrol-2] [INFO] [1765164372.100411075] [patrol]: Switch True
[patrol-2] [INFO] [1765164372.101321312] [patrol]: LengthTest
[patrol-2] [INFO] [1765164372.102738669] [patrol]: distance: 0.3024302162352
4424
[patrol-2] [INFO] [1765164372.600518239] [patrol]: Switch True
[patrol-2] [INFO] [1765164372.601291384] [patrol]: LengthTest
[patrol-2] [INFO] [1765164372.602654403] [patrol]: distance: 0.4165993920396
192
[patrol-2] [INFO] [1765164373.101332018] [patrol]: Switch True
[patrol-2] [INFO] [1765164373.102755968] [patrol]: LengthTest
[patrol-2] [INFO] [1765164373.104203150] [patrol]: distance: 0.5255122298142
755
[patrol-2] [INFO] [1765164373.601116357] [patrol]: Switch True
[patrol-2] [INFO] [1765164373.602500273] [patrol]: LengthTest
[patrol-2] [INFO] [1765164373.604522642] [patrol]: distance: 0.6358100060346
081
```

- If there is an obstacle on the patrol path, the robot will stop and display **obstacles**



3. Source Code Analysis

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/lidar/laser_patrol.py
```

The patrol node uses a **timer-driven + state machine** architecture; the core logic is as follows:

1. **Dynamic parameter loading**: each cycle, the `on_timer` callback first reads all configurations from the parameter server (`Switch`, `Command`, `ResponseDist`, `Linear`, `Angular`, `Length`, `LaserAngle`, `LineTolerance`, `RotationTolerance`, etc.), supporting real-time adjustment via `rqt_reconfigure`.
2. **Patrol route dispatch**: when `Switch=True`, selects the corresponding patrol mode based on the `Command` string:
    - `LengthTest`: straight-line patrol, calls `advancing(self.Length)` to advance a specified distance
    - `Circle`: circle patrol, calls `Spin(360)` to rotate 360° in place
    - `Square`: square patrol, calls `Square()` to execute the four-sides-four-corners sequence
    - `Triangle`: triangle patrol, calls `Triangle()` to execute the three-sides-three-corners sequence
3. **Task completion handling**: after each patrol function returns `True`, it automatically sets `Switch` to `False` and `Command` to `"finish"`, entering the stopped state.
4. **Cyclic patrol logic**: when `Switch=False` and `Command="finish"`:
    - If `Set_loop=True`: restores `Command` to the original route (`command_src`), sets `Switch` back to `True`, and continues the next round of patrol
    - If `Set_loop=False`: remains in the stopped state and does not restart automatically

> **Obstacle avoidance mechanism**: each patrol function (`advancing`, `Spin`, `Square`, `Triangle`) continuously calls LiDAR detection internally. When the front obstacle's distance ≤ `ResponseDist` and its angle is within the ±`LaserAngle` range, it immediately stops the car and triggers the buzzer, and continues execution only after the obstacle is moved away.

```
    def on_timer(self):
        # print("self.error: ",self.error)
        self.Switch = self.get_parameter("Switch").get_parameter_value().bool_value
        self.Command = self.get_parameter("Command").get_parameter_value().string_value
        self.Set_loop = self.get_parameter("Set_loop").get_parameter_value().bool_value
        self.ResponseDist = (
            self.get_parameter("ResponseDist").get_parameter_value().double_value
        )
        self.Linear = self.get_parameter("Linear").get_parameter_value().double_value
        self.Angular = self.get_parameter("Angular").get_parameter_value().double_value
        self.Length = self.get_parameter("Length").get_parameter_value().double_value
        self.LaserAngle = (
            self.get_parameter("LaserAngle").get_parameter_value().double_value
        )
        self.LineTolerance = (
            self.get_parameter("LineTolerance").get_parameter_value().double_value
        )
        self.RotationTolerance = (
            self.get_parameter("RotationTolerance").get_parameter_value().double_value
        )
        self.RotationScaling = (
            self.get_parameter("RotationScaling").get_parameter_value().double_value
        )

        index = 0

        if self.Switch == True:
            index = 0
            self.get_logger().info("Switch True")
            if self.Command == "LengthTest":
                self.command_src = "LengthTest"
                self.get_logger().info("LengthTest")
                advancing = self.advancing(self.Length)
                if advancing == True:
                    self.Switch = rclpy.parameter.Parameter(
                        "Switch", rclpy.Parameter.Type.BOOL, False
                    )
                    all_new_parameters = [self.Switch]
                    self.set_parameters(all_new_parameters)
                    self.Command = rclpy.parameter.Parameter(
                        "Command", rclpy.Parameter.Type.STRING, "finish"
                    )
                    all_new_parameters = [self.Command]
                    self.set_parameters(all_new_parameters)

            elif self.Command == "Circle":
                self.command_src = "Circle"
```

```
                spin = self.Spin(360)
                if spin == True:
                    self.get_logger().info("spin done")
                    # self.Command = "finish"
                    self.Switch = rclpy.parameter.Parameter(
                        "Switch", rclpy.Parameter.Type.BOOL, False
                    )
                    all_new_parameters = [self.Switch]
                    self.set_parameters(all_new_parameters)
                    self.Command = rclpy.parameter.Parameter(
                        "Command", rclpy.Parameter.Type.STRING, "finish"
                    )
                    all_new_parameters = [self.Command]
                    self.set_parameters(all_new_parameters)

            elif self.Command == "Square":
                self.command_src = "Square"
                square = self.Square()
                if square == True:
                    self.Command = rclpy.parameter.Parameter(
                        "Command", rclpy.Parameter.Type.STRING, "finish"
                    )
                    all_new_parameters = [self.Command]
                    self.set_parameters(all_new_parameters)

            elif self.Command == "Triangle":
                self.command_src = "Triangle"
                triangle = self.Triangle()
                if triangle == True:
                    self.Command = rclpy.parameter.Parameter(
                        "Command", rclpy.Parameter.Type.STRING, "finish"
                    )
                    all_new_parameters = [self.Command]
                    self.set_parameters(all_new_parameters)

        else:
            self.pub_cmdVel.publish(Twist())
            self.pub_beep.publish(UInt16(data=0))
            self.get_logger().info("Switch False")
            if self.Command == "finish":
                self.get_logger().info("finish")
                if self.Set_loop == True:
                    self.get_logger().info("Continute")
                    self.Command = rclpy.parameter.Parameter(
                        "Command", rclpy.Parameter.Type.STRING, self.command_src
                    )
                    all_new_parameters = [self.Command]
                    self.set_parameters(all_new_parameters)
                    self.Switch = rclpy.parameter.Parameter(
                        "Switch", rclpy.Parameter.Type.BOOL, True
                    )
                    all_new_parameters = [self.Switch]
                    self.set_parameters(all_new_parameters)
```

```
                else:
                    self.get_logger().info("Not loop")
                    self.Switch = rclpy.parameter.Parameter(
                        "Switch", rclpy.Parameter.Type.BOOL, False
                    )
                    all_new_parameters = [self.Switch]
                    self.set_parameters(all_new_parameters)
```